Corgi Raytracing

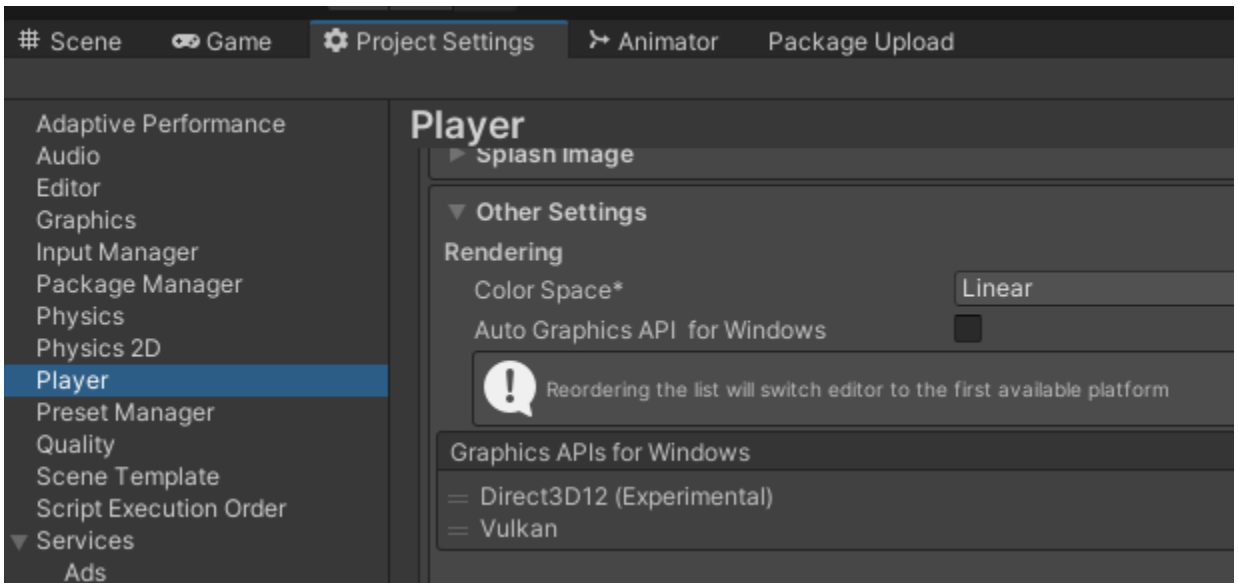
ReadMe v1.5.2

[latest documentation here]

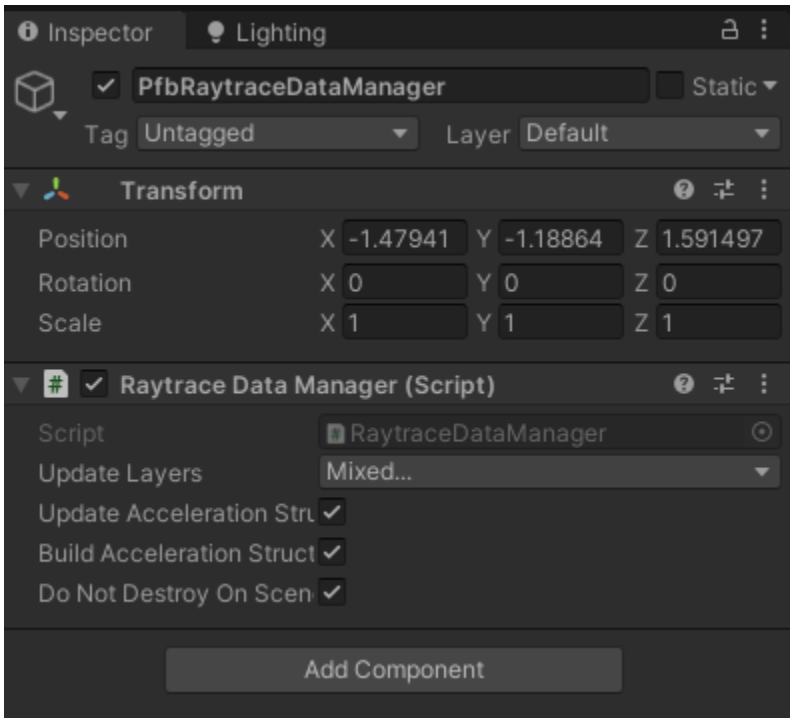
Thanks for purchasing Corgi Raytracing! This document will help you get started, quickly. It also provides some tips for more advanced usage.

Getting Started (All Pipelines)

Note: For Windows, Unity must be in DX12 mode.



The very first thing you need to do is locate the prefab PfbRaytraceDataManager and place it into the scene. This manager handles updating Unity's RayTracingAccelerationStructure. Without it, the other components cannot function properly. It's recommended to either place one in every scene with "DoNotDestroyOnSceneChange" disabled, or to place it in your startup scene with that option enabled. Or, you can keep that option disabled, and manage its lifetime yourself, manually.



Builtin Render Pipeline Effects

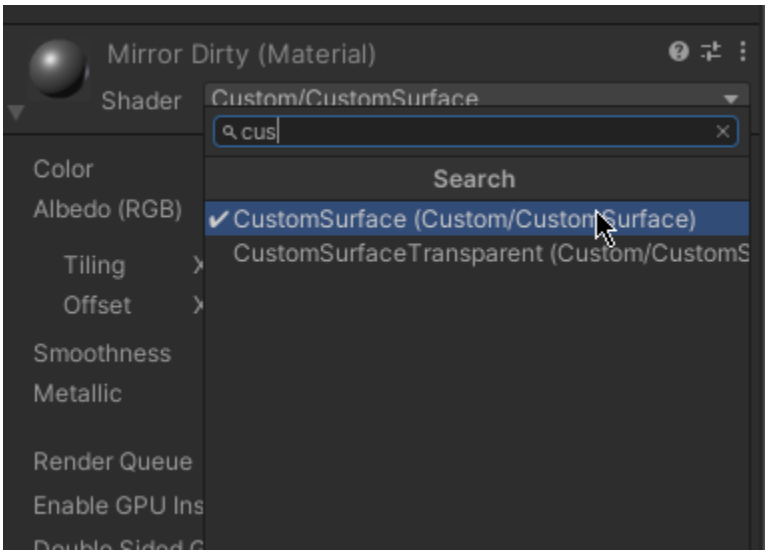
The following will describe how to add the raytracing effects for the built-in render pipeline.

Raytraced Reflections

On your game camera, add the component RaytraceReflectionsManager. When this script is attached, you should immediately see reflections in your scene, assuming you have highly reflective surfaces visible, that are using either the custom standard shader, or a custom shader modified with added raytracing passes.

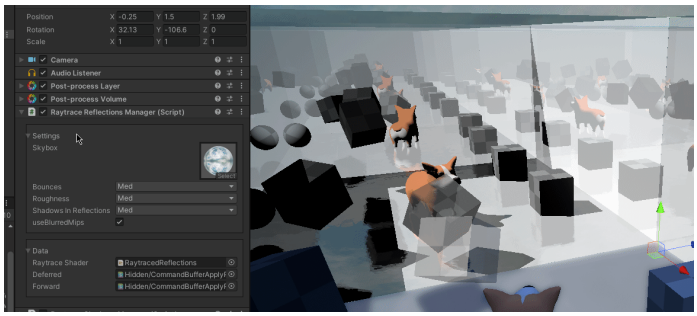
If, in the likely scenario, you are still using the Standard shader or some custom shaders, you will not yet see the reflections. For our raytraced reflections to work, ALL shaders must either be modified or replaced, such that they include a Raytraced Reflection pass.

Corgi Raytracing comes with some standard shader replacements, so you can get started quickly.

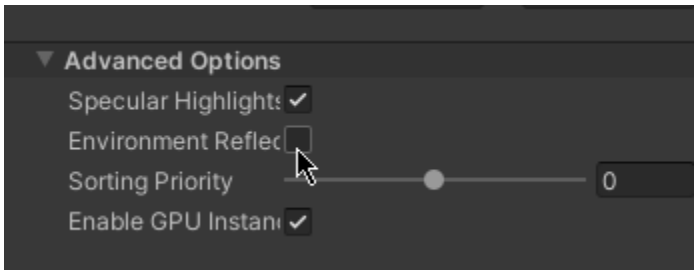


Try replacing your standard shaders with one of these variants, to get started.

- Corgi/Surface
- Corgi/Surface (Specular)

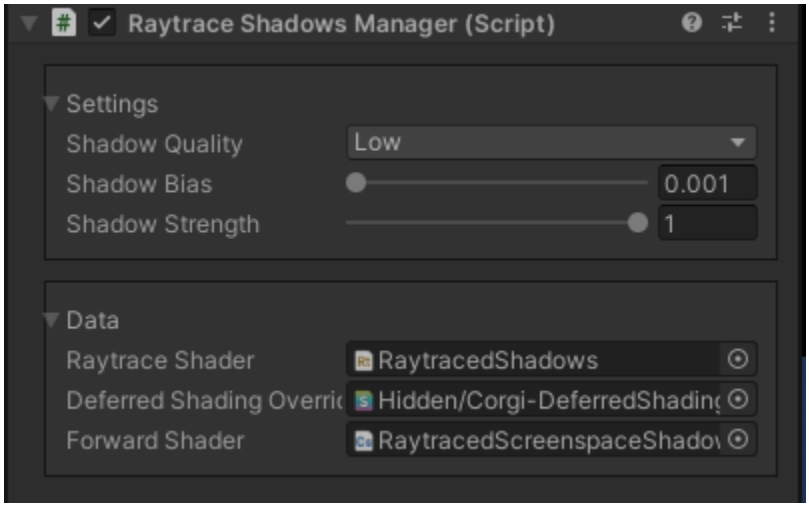


For URP, when using Raytraced Reflections, you should disable the Environmental Reflections toggle in your Lit materials. Otherwise, you will have URP's reflections (skybox) overlaid on top of the RTX reflections, which looks pretty bad.



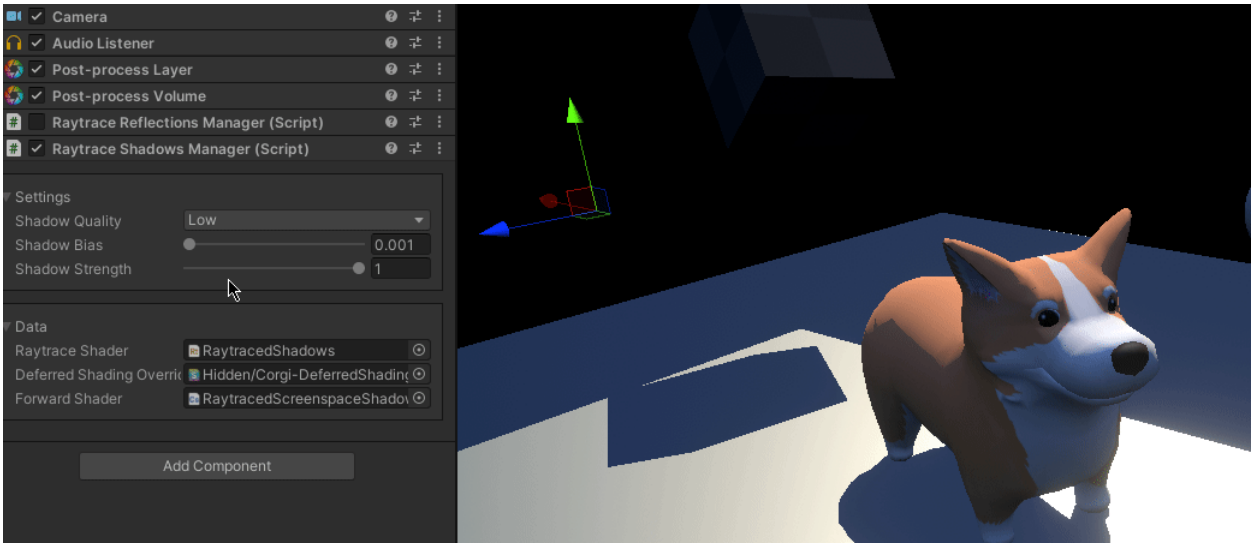
Raytraced Shadows

On your game camera, add the component RaytraceShadowsManager. Similar to the reflections component, for this effect to work, your shaders must contain the appropriate raytrace pass.



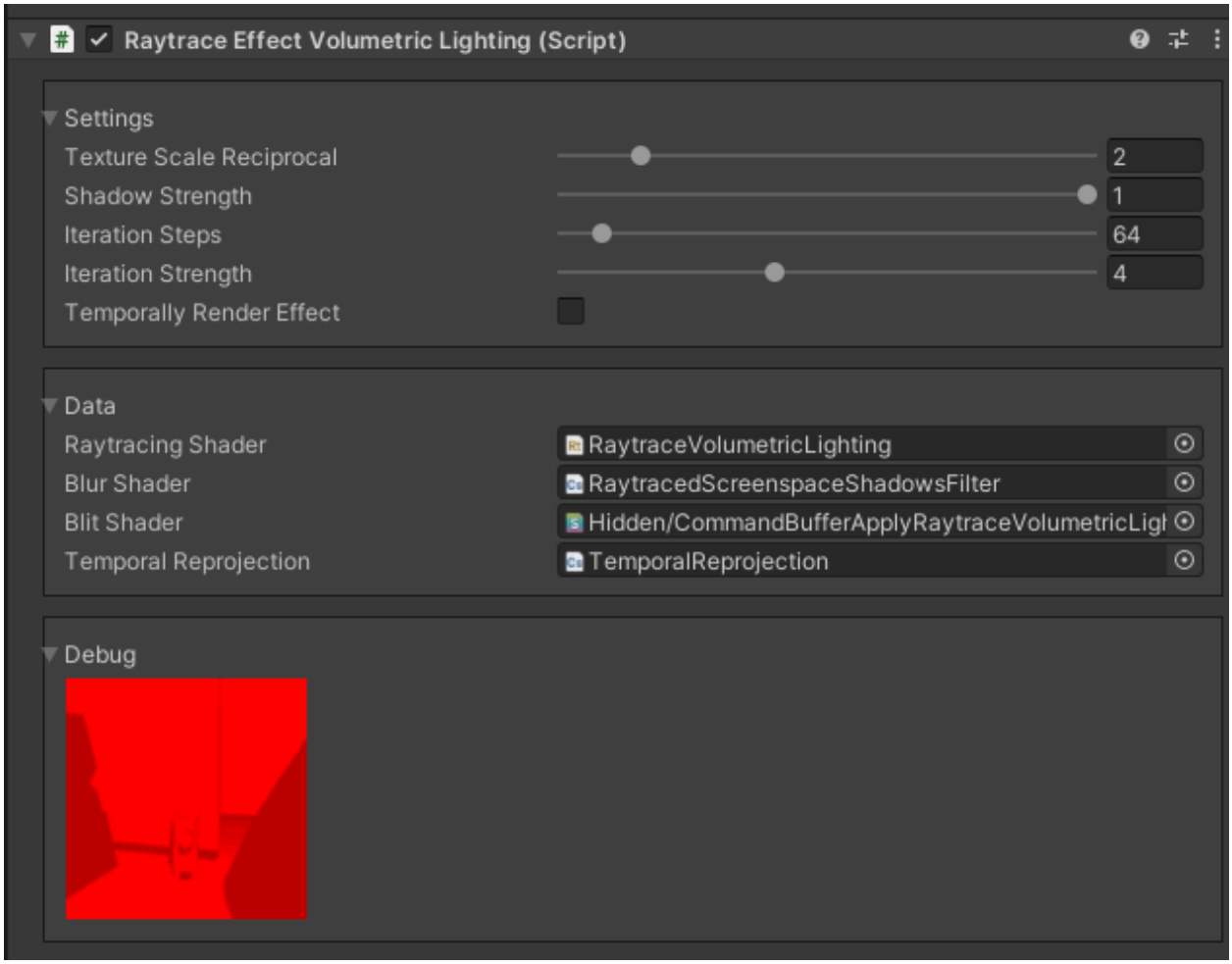
You can quickly get started by just replacing your standard shaders with our builtin replacement.

Or, you can add in the appropriate fallback to quickly add support to existing shaders.



Raytraced Volumetric Lighting

On your game camera, add the component RaytraceEffectVolumetricLighting. Similar to the others, for this effect to work, your shaders must contain the appropriate raytrace pass.

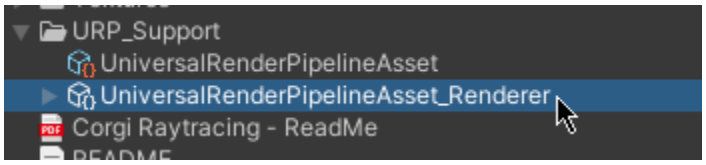


Increasing the number of iteration steps will have a direct impact on performance. The more steps there are, the better the effect will look, but at the cost of performance. You can cheat a bit by turning up the texture scale reciprocal (the higher this is, the lower quality the output texture will be for this effect).

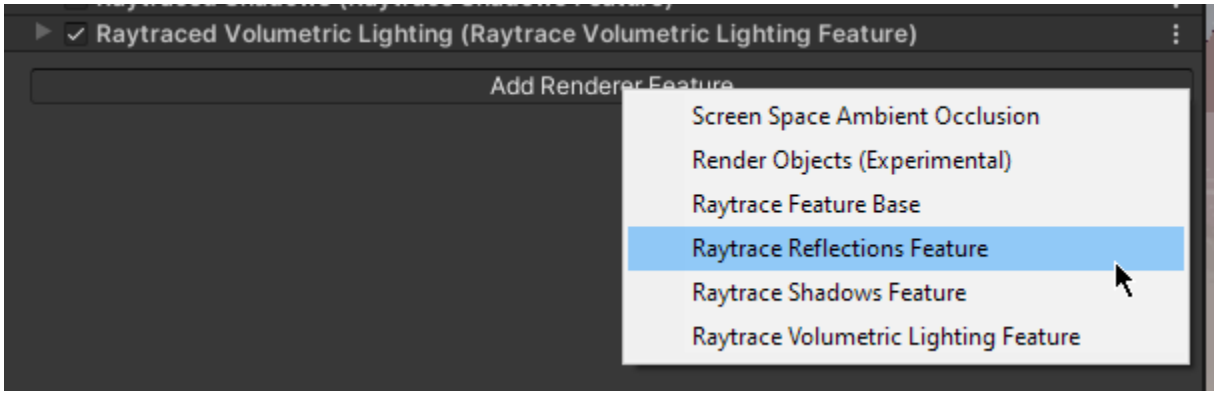
Universal Render Pipeline Effects

The effects described in the previous section are also possible to use in URP, however the setup is different. You will still need the PfbRaytraceDataManager in your scene, however you do not need to put anything directly on your Camera gameobject.

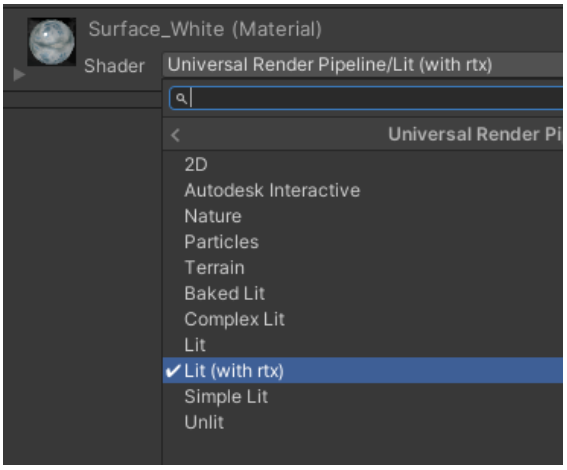
In URP, Corgi Raytracing relies on the Feature/Pass functionality of URP. You can see an example URP Renderer setup in the plugin here:



Feel free to look at how these are setup in order to add these effects to your personal URP renderer.



The settings in these effects will mostly match what you see in the previous section's documentation of these effects. There are some limitations in URP support for this plugin. For example, URP does not yet support motion vector rendering, and so Corgi Raytracing does not yet allow for temporal reprojection of the effects.



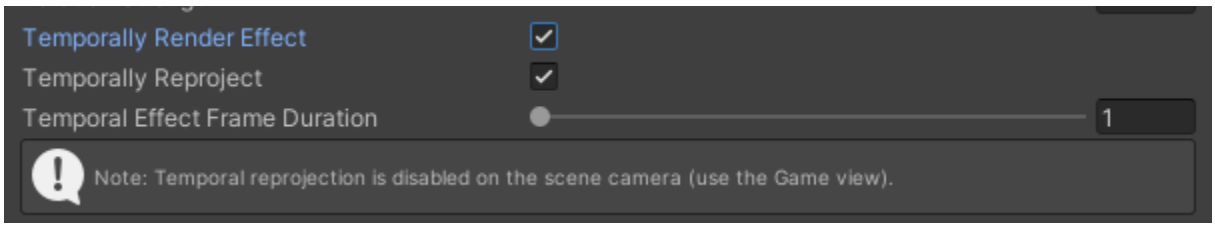
Try replacing your Lit shaders with one of these variants, to get started.

- Universal Render Pipeline/Lit (with rtx)

If you want to use a shadergraph shader with raytracing, you'll need to export the shader to a .shader file and add a fallback, as described here.

Temporal Rendering of the Effects

All of the included raytracing effects have the ability to be temporally rendered through the use of dithered rendering with optional temporal reprojection.



Temporal Rendering

When I say temporal rendering, all I mean is that the effect takes multiple frames to “catch up” and have the whole screen rendered. This is achieved by skipping every other ray per-frame, when generating rays for the effect. When the temporal effect frame duration is set to a higher value, the number of pixels skipped each frame increases. You should see a nearly 2x performance improvement for turning this effect on at setting = 1. Increasing it to 2 is almost another 2x improvement to the performance of the effect. However, higher values result in noisier and thus nastier results in the image. If your effect is highly blurred, these results may be acceptable in your use-case.

Temporal Reprojection

To cover up the noise and general nastiness, I’ve implemented some naive temporal reprojection. After motion vectors have been rendered, an additional pass is done to project the pixels from the current frame to a guestimated position of where they will be next frame. The goal is to move the dithered pixels so that they match up with where they need to be next frame based on movement from this frame. This effect only works “correctly” when the temporal effect frame duration is set to 1. If it is set higher, you will get noise-y results. If this is acceptable to you because of your use case, feel free to do so.

Temporal Reprojection seems to have issues in Unity’s scene view cameras (due to motion vectors being incorrect there) - so I have disabled this ability when the effect is rendered in the Scene view.

Also - you may need to enable/disable the component when tweaking settings while temporal reprojection is on. I’ll try to improve this in the future.

Shader Passes (Builtin)

In order to use this plugin, you must touch every shader in your project. It's a pain, but it's the best way to get the best results.

If you are only using the Standard shader, this isn't that big of a pain. You just need to replace Unity's Standard shader with ours.

If you are using Custom shaders, I've tried to make this as effortless as possible. All you need to do is open up your .shader files, and add in the following line after your final SubShader pass:

```
FallBack "Corgi/Raytracing/StandardFallbackRaytrace"
```

```
Shader "Custom/CustomSurface"
{
    Properties
    {
        _Color ("Color", Color) = (1,1,1,1)
        _MainTex ("Albedo (RGB)", 2D) = "white" {}
        _Glossiness ("Smoothness", Range(0,1)) = 0.5
        _Metallic ("Metallic", Range(0,1)) = 0.0
    }

    // surface
    SubShader
    {
        Tags { "RenderType"="Opaque" }
        LOD 200

        CGPROGRAM

        #pragma surface surf Standard fullforwardshadows
        #pragma target 3.0

        sampler2D _MainTex;

        struct Input
        {
            float2 uv_MainTex;
        };

        half _Glossiness;
        half _Metallic;
        fixed4 _Color;

        void surf (Input IN, inout SurfaceOutputStandard o)
        {
            // Albedo comes from a texture tinted by color
            fixed4 c = tex2D (_MainTex, IN.uv_MainTex) * _Color
            o.Albedo = c.rgb;
            // Metallic and smoothness come from slider variabl
            o.Metallic = _Metallic;
            o.Smoothness = _Glossiness;
            o.Alpha = c.a;
        }

        ENDCG
    }

    FallBack "Corgi/Raytracing/StandardFallbackRaytrace"
}
```

A full custom shader might look like this screenshot.

If you already have a fallback, you need to replace it with this one. If your existing fallback is "Diffuse," you don't need to do anything else. It should just work, because there is a Diffuse fallback within this fallback.

However, if you need to use the fallback passes from something other than Diffuse, you may need to think about implementing these raytracing passes manually.

I've provided a few cginc files with some helpful macros, for writing your own raytracing shaders.

Many of these ideas are borrowed from Unity's HDRP implementation, so I highly recommend downloading their raytracing shader source code and browsing through it. Here's a link.

Manually Adding Raytracing Shader Passes

Please refer to the shader `Corgi/Raytracing/StandardFallbackRaytrace`. Most of the time, you can get away with just copy/pasting out these Passes into your own custom shaders.

```
Pass
        {
            Name "RaytraceReflectionPass"

            HLSLPROGRAM

            #define RAYTRACE_STANDARD_HIT
            #pragma raytracing Standard_ClosestHit_Generic
            #include "../Includes/RaytracedReflectionHitPass.cginc"

            ENDHLSL
        }

        Pass
        {
            Name "RaytraceShadowsPass"

            HLSLPROGRAM

            #pragma raytracing Standard_ClosestHit_ShadowsGeneric
            #include "../Includes/RaytracedShadowsHitPass.cginc"

            ENDHLSL
        }
```

The magic happens inside the includes, but these tiny passes make it really easy to add corgi raytracing support to any of your shaders. Let’s look inside the includes.

RaytracedReflectionHitPass.cginc

```
[shader("closesthit")]
void Standard_ClosestHit_Generic(inout ReflectionPayload payload :
SV_RayPayload, AttributeData attributes :
SV_IntersectionAttributes)
{
    // ....
}
```

This is the core of any of these raytracing effects. When a ray has finished intersecting with objects and has found its closest intersection, this kernel is executed.

As you can see inside, I've provided some helpful macros, in case you want to write your own raytracing effects, using Corgi Raytracing as a base.

```
    Vertex v0, v1, v2;
    GetVertexData(v0, v1, v2);
```

This is what it sounds like, it fetches the three vertices which make up the triangle the ray intersected.

```
float3 position = INTERPOLATE_RAYTRACING_ATTRIBUTE(v0.position,
v1.position, v2.position, barycentricCoords);
```

This uses the data available to interpolate vertex data. It's similar to what happens when you are using interpolated data from a vertex shader in a fragment shader. If you do not want to interpolate, I've also provided a `NOINTERPOLATE_RAYTRACING_ATTRIBUTE` macro.

```
BounceRay(worldPosition, rayDirection, _Glossiness, _Metallic,
payload);
```

This is a helper function for reflections, which instantiates one or more rays which may call more of these closesthit kernels.

RaytracedShadowsHitPass.cginc

This file is similar in structure to the reflections variant. The only new addition here is this function:

```
Cast(rayPosition, rayDirection, payload);
```

This is used for spawning rays, like the previous BounceRay function.

To get Corgi Raytracing to work in both deferred and forward, some things may seem redundant in this shader code.

- In Forward, rays are spawned from the perspective of the camera, and cast out to look for intersections.
- In Deferred, the gbuffers are sampled and rays are cast out from the calculated world space, to look for intersections.

So, while in deferred this intersection pass is only used for determining if a previous ray hit “something,” while in forward we need to first cast to find something to then spawn another ray, which is why we may also cast here.

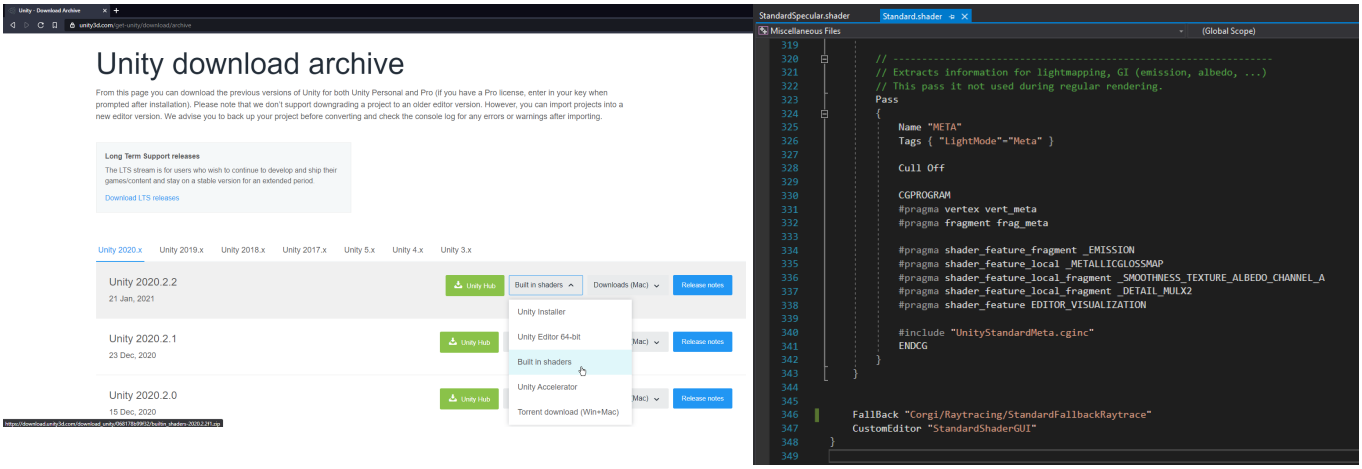
Determining what “kind” of ray we’re dealing with is handled by a simple variable `rayType` in the ray’s payload.

Updating a Built-in Unity shader to be Corgi Raytracing compatible

I've included the Standard shader, but most of the Unity built-in shaders can be somewhat compatible with Corgi Raytracing with a simple tweak. This guide can help you if you update Unity and the included Standard shader replacement stops working, or if you want to try Raytracing against another Unity built-in shader.

To convert almost any of Unity's built-in shaders to be compatible, follow these simple steps.

1. Download the built-in shaders for your version of Unity from the Unity Archive here: https://unity3d.com/get-unity/download/archive
2. Extract and navigate to <BuiltInShaders>/DefaultResourcesExtra. Find the shader you care about, and open it in your favorite text editor. You can use Visual Studio, Sublime, whatever.
3. Scroll to the bottom. Replace the Fallback line with the following: `FallBack "Corgi/Raytracing/StandardFallbackRaytrace"`
4. Scroll to the top. Change the name to make it Unique. Something like Corgi/X or Raytracing/X.
5. Save it as a new file, located in your project. It should now appear in the list of shaders when changing the shader of a material in Unity!



Shader Passes (URP)

In order to use this plugin, you must touch every shader in your project. It's a pain, but it's the best way to get the best results.

If you are only using URP's Lit shader, this isn't that big of a pain. You just need to replace Unity's Lit shader with ours.

If you are using Custom shaders, I've tried to make this as effortless as possible. All you need to do is open up your .shader files, and add in the following line after your final SubShader pass:

```
FallBack "Corgi/Raytracing/URPFallbackRaytrace"
```

Manually Adding Raytracing Shader Passes

Please refer to the shader `Corgi/Raytracing/URPFallbackRaytrace`. Most of the time, you can get away with just copy/pasting out these Passes into your own custom shaders.

```
Pass
        {
            Name "RaytraceReflectionPass"

            HLSLPROGRAM

            #pragma raytracing URP_ClosestHit
            #include
"../Includes/URP/RaytracedReflectionHitPass_URP.cginc"


            ENDHLSL
        }

        Pass
        {
            Name "RaytraceShadowsPass"

            HLSLPROGRAM

            #pragma raytracing URP_ClosestHit_Shadows
            #include
"../Includes/URP/RaytracedShadowsHitPass_URP.cginc"

            ENDHLSL
        }
```

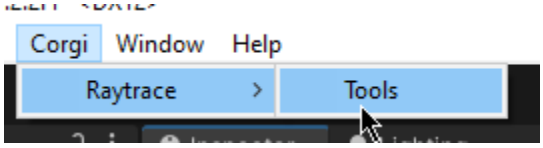
The magic happens inside the includes, but these tiny passes make it really easy to add corgi raytracing support to any of your shaders. Let's look inside the includes.

The URP variants of `RaytraceReflectionPass` and `RaytraceShadowsPass` are basically the same as the builtin variants, with a few key variables being renamed to allow for proper textures and whatnot to be sampled correctly.

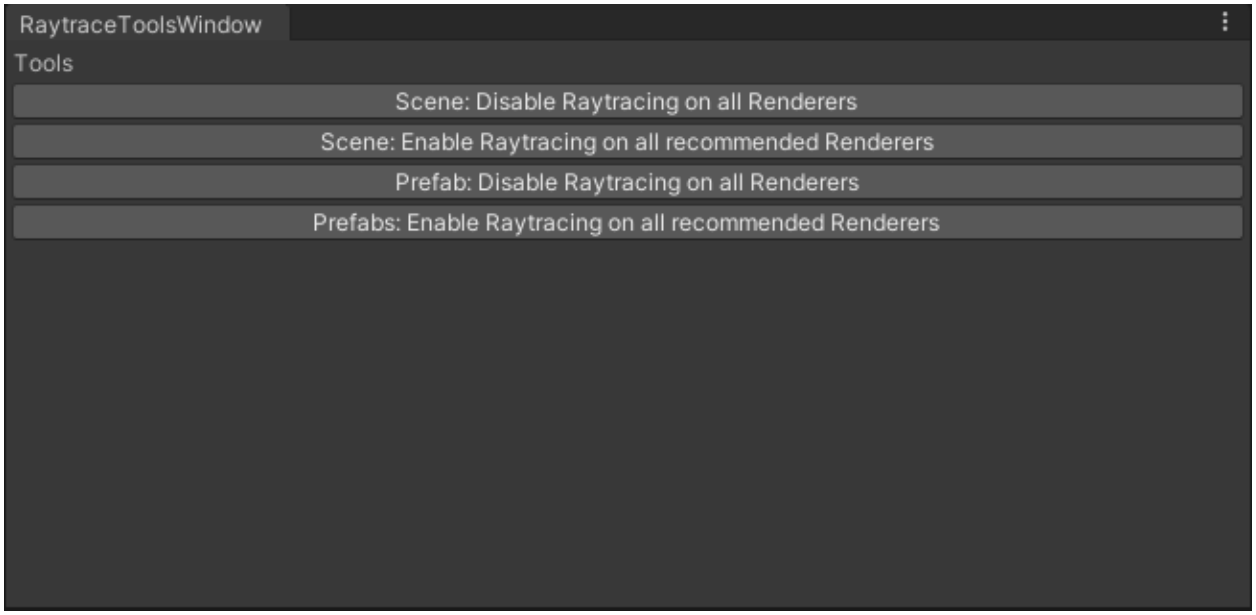
Tools

I've included a few helper tools for managing ray tracing in an existing project. I highly recommend disabling RaytraceMode on the vast majority of your renderers, for performance reasons.

To access these tools, use the toolbar at the top of Unity -> Corgi -> Raytrace -> Tools.



If you have any suggestions for some useful tools related to the plugin, shoot me an email and I'll consider adding them to this window.



Platform Limitations

Feature support table:

	Builtin (Deferred)	Builtin (Forward)	URP (forward)	HDRP
Reflections	Yes	Yes[1]	Yes[1]	No
Shadows	Yes	Yes	Yes	No
Volumetric Lighting	Yes	Yes	Yes	No
Temporal Rendering	Yes	Yes	Yes[2]	No
Temporal Reprojection	Yes	Yes	No[2]	No

[1] In forward rendering, using a blurred mip chain to sample the reflections for cheap roughness estimation is not currently supported.
[2] URP does not yet have a built-in motion vector solution, so temporal reprojection of temporally dithered rendering is not yet available in this plugin.

Please understand, there are some limitations with this raytracing implementation.

- In Forward rendering, there are no gbuffers. As such, there is not a straightforward way to get (cheap blurred) smoothness for the first ray (to use the mips with the smoothness slider). This means that deferred will have higher quality reflections for less cost, in this regard.
- Animated vertex shaders are not supported in our implementation. This may come in the future.
- Skinned Mesh Renderers only work in 2020.1 and above. They do not work in 2019.4. This is a unity limitation.
- Temporal reprojection does not work in the scene view.
- Architecture in the build settings must be set to **x86_64**. If it is set to x86, some features may not work.
- Builtin **does not** have SPI support for VR. However, URP **does** have SPI support for VR.

Performance Tips

- By default, turn off Raytracing for all renderers in Unity. When something is statically batched, be sure to mark it's raytracing as static, too.
- Only enable Raytracing on things that are actually visible. The less things in the RaytraceAccelerationStructure, the better your performance can be.
- Try to use smaller meshes to raytrace against, so the AABB testing will allow skipping over most meshes.
- If you do not need or care about dynamic objects for raytraced effects, uncheck the update options on the RaytraceDataManager.
- Use layers!
- Use short rays, where possible!

I've included the window tool for helping manage assets for raytracing settings on renderers. In the Unity toolbar at the top, check out: `Corgi/Raytrace/Tools`

Need Help?

If anything is unclear, please email me at coty@wanderingcorgi.com - I will try to get back to you as soon as possible. If you have any suggestions or feature requests, I'm open to implementing them! By the way, this email is forwarded to my personal email, which I'll send replies from.